



Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Named Entity Recognition in German-language Telegram channels

Bachelor Thesis

B.Sc. Computer Science and Business Administration

Faculty 4

by

Elisabeth Steffen

Berlin, August 19, 2022

1st Supervisor: Prof. Dr. Helena Mihaljević

2nd Supervisor: Prof. Dr. Theresa Busjahn

Abstract

This thesis investigates the potentials and limitations of Named Entity Recognition (NER) approaches for the automated analysis of text data from the messenger service Telegram.

The thematic focus is on the mobilizations for protests against the COVID-19 measures in Germany. The rise of conspiracy theories in the context of the COVID-19 pandemic not only reinforced antisemitic and racist discrimination against certain communities, but also generated an increasingly toxic and violent attitude towards political decision-makers and scientific experts.

The past months have once again shown the importance of investigative work by critical journalists and non-governmental organizations to critically monitor the corresponding mobilization strategies on Telegram. A central challenge here is the large amounts of user-generated data from often very dynamic contexts, which are usually analyzed manually. Here, approaches from the field of natural language processing can offer a valuable addition.

Against this background, this thesis aims to evaluate the applicability of Named Entity Recognition for the investigation of the COVID-19 protest movement in German-language Telegram channels, focusing on entities representing people, places, organizations, actions, dates, and times.

This work contributes to the development of German-language Named Entity Recognition in general by applying this approach to a specific text genre and domain, and provides first insights for developing an NER tool for researchers and non-governmental organizations in the future.

Contents

1	Introduction	1
1.1	Background	1
1.2	Objective	5
1.3	Thesis Structure	5
2	Theory	6
2.1	Natural Language Processing	6
2.2	Information Extraction	7
2.3	Named Entity Recognition	9
2.3.1	Common entity types	9
2.3.2	IOB tagging scheme	10
2.3.3	Evaluation of NER systems	11
2.3.4	Challenges for NER	12
2.3.5	Specific Challenges for NER in User-Generated Texts .	15
2.3.6	Important Shared Tasks and Datasets for German-Language NER	17
2.3.7	Important Experimental Studies on NER on Social Me- dia Texts	19
2.3.8	Important Shared Tasks and Datasets for NER on So- cial Media Texts	20
2.4	Overview of NER approaches	21
2.4.1	Rule-based and Knowledge-based Approaches	21
2.4.2	Machine-learning Based Approaches	22
2.4.3	The Introduction of Neural Networks	24
2.4.4	Recurrent Neural Networks	25
2.4.5	Transformers	26
2.5	Summarization	27

3	Qualitative Exploration of Existing Solutions	28
3.1	German-language Named Entity Recognition with spaCy and Flair	29
3.1.1	The German-language spaCy model	29
3.1.2	The German-language standard NER Flair model	30
3.1.3	Example Data	31
3.1.4	Implementation	31
3.1.5	Results	32
3.2	Exploring the Potentials of Machine Translation for NER	34
3.2.1	The English-language spaCy model	35
3.2.2	The English-language Flair model	36
3.2.3	Implementation	36
3.2.4	Results	37
3.3	Summarization	37
4	Towards building a custom model for NER in Telegram channels	39
4.1	Data	39
4.1.1	Existing German-language datasets for this specific task	39
4.1.2	Domain-specific data from Telegram channel	40
4.1.3	Text cleaning	41
4.1.4	Sample Selection	44
4.2	Annotation scheme	48
4.3	Annotated dataset	49
4.4	Baseline	49
4.5	Models	49
4.6	Building a Pipeline	49
4.6.1	Tokenization	50
4.6.2	Vectorization?	50
4.6.3	Model heads	52
4.6.4	Post-processing	52
4.7	Evaluation	53
4.8	Hyperparameter tuning	53
4.9	Processing of results	53
4.10	Summarization	53
5	Results	54

6	Conclusion	55
6.1	Summarization	55
6.2	Discussion	55
	Bibliography	61
.1	Named Entity - Basic Definition	66
.2	Definition of Different Entity Types	67
.3	Specifications	67
.3.1	Mixed-language messages	67
.3.2	Incomprehensible messages	69
.3.3	Short messages	69
.3.4	Punctuation	69
.4	Detailed Annotation Instructions for Entity Types	70
.4.1	PERSON	70
.4.2	LOCATION	71
.4.3	ORGANIZATION	72
.4.4	TIME	73
.4.5	DATE	73
.4.6	ACTION	74

Chapter 1

Introduction

1.1 Background

The outbreak of the COVID-19 pandemic was followed by an increasing spread of misinformation and conspiracy theories. Those narratives evolved not only around the virus itself, but also about the intentions of those responsible for putting into effect measures for its containment. Those measures included the mandatory use of face masks, rapid testing, and vaccinations, at times functioning as admission requirements in the workplace or publicly accessible spaces such as restaurants and theatres. Furthermore, unprecedented restrictions were imposed on public life through several general or partial lock-downs of social and economic life, as well as temporary bans of gathering in public. These restrictions also heavily affected the possibilities to hold political demonstrations. From the beginning on, these measures were accompanied by critique and protests from various political spectres. In Germany, the so-called ‘Querdenken’ (lateral thinking) movement soon became the most prominent network organizing protests against the COVID-19 containment measures. The movement reached its preliminary climax with several large demonstrations taking place in Berlin, directed against legislative changes regarding the containment of the pandemic passed by the

German parliament. In the context of these demonstrations, violent clashes occurred between police and protesters, which often refused to act according to the official restrictions such as wearing a face mask. Furthermore, there were several incidents in which journalists were violently attacked by protesters and accused of being part of the ‘Lügenpresse’ (lying press). Furthermore, there were two incidents in which the seat of the German parliament, the Reichstag, was targeted by protesters.

On April 21st 2021, the last large authorized demonstration of the ‘Querdenken’ movement took place in Berlin. In response to several violations of official restrictions by demonstration participants, police decided to disperse the demonstration. Several protesters subsequently moved to other areas nearby where clashes between protesters and police continued to occur throughout the rest of the day.

The subsequent demonstrations announced by the ‘Querdenken’ movement to take place in Berlin were prohibited by authorities, arguing with the experience at prior demonstrations that participants would act against the official restrictions. In spite of the ban, several hundred protesters gathered in different spots in Berlin e.g. on the 21st, 28th and 29th of August 2021.

The mentioned bans of large demonstrations of the ‘Querdenken’ movement as well as the general restrictions for demonstrations and public gatherings (such as limiting the number of participants to a maximum of twenty persons), decreased the momentum of the large, centralized protests focussing on the German legislative located in Berlin. However, the protests did not stop as a result, but rather changed their character: They became more decentralized and a shift to more informal forms of gatherings and protests could be observed: In several German cities and towns, people gathered for ‘Mahnwachen’ or ‘Spaziergänge’ (walks) to protest against the government’s COVID-19 politics. Since going for a walk individually was a legal outdoor activity even under the strictest lock-down regulations, this type of action

was chosen to circumvent the restrictions imposed on demonstrations. In effect, these walks ('Spaziergänge') often resulted in large gatherings of people moving through the streets together, often holding signs or chanting slogans against German politicians or the COVID-19 containment measures in general.

On some occasions, these walks turned into gatherings in front of the private homes of politicians in charge for the COVID-19 measures on federal state level: In December 2021, around thirty members of the far-right group 'Freie Sachsen' (Free Saxonians) gathered in front of the home of Saxony's Minister of Health, Petra Köpping, carrying torchlights. The incident received a lot of public attention, and was criticized in the media as well as by German politicians as attempts to intimidate politicians under the pretext of the protesters right to 'freedom of speech'. Earlier that year, the same group had gathered in front of Saxony's prime minister, Michael Kretschmer, who is one of the most prominent targets of the radical movement against the COVID-19 containment measures. In December 2021, an investigative report by German journalists revealed that members of the Telegram chat group 'Dresden Offlinevernetzung' had talked about concrete preparations to assassinate Kretschmer. And a few months later, by mid-April 2022, police foiled a plot to kidnap Germany's Minister of Health Karl Lauterbach and to sabotage power facilities in order to create chaos which would ultimately allow to overthrow the government. The rise of conspiracy theories in the context of the COVID-19 pandemic has thus not 'only' intensified hate-speech against and the discrimination of certain communities in the form of antisemitism and racism - this process was accompanied by an increasingly toxic and potentially violent climate for politicians in Germany.

The messenger service Telegram played a key role in the mobilization of the movement. The service allows the creation of large groups and channels to disseminate information quickly to a large number of users. Unlike

Facebook or Google, which hand over data to authorities at their request, Telegram does not cooperate with authorities. This ensures a high degree of anonymity for its users which has proven beneficial for protest movements in authoritarian countries. This anonymity poses a barrier to political repression and criminal prosecution. This makes the service attractive not only for social movements, but also for criminal and terrorist activities, and also facilitates the spread of hate-speech and calls for violence like those mentioned above.

The evolvement of the COVID-19 protests in Germany clearly demonstrate how online and offline contexts influence and often reinforce each other. But even though non-governmental organizations such as e.g. CEMAS had warned about the antidemocratic and violent potential of parts of the movement, and had pointed out the involvement of political activists from Germany's extremist far-Right, German authorities underestimated the dangerous potential of this movement for a long time. Only after journalists had revealed the plans to murder Michael Kretschmer, and after several gatherings in front of private homes of politicians had taken place, the issue became more of a serious concern.

This points out the relevance of investigative work by critical journalists and non-governmental organizations to monitor the mobilization and networking strategies of movements such as 'Querdenken' in Germany. However, while these actors often have to deal with large amount of data, their investigation often relies on manually conducted research and data evaluation. Machine Learning-based approaches for Natural Language Processing (NLP) are often not easily accessible for these actors, making it hard for them to process large amounts of data from the often very dynamic contexts of informal political networks.

1.2 Objective

This thesis aims at exploring the potentials of a particular NLP technology, namely Named Entity Recognition (NER) for the investigation of the mobilization for protests against COVID-19 measures in a German-language Telegram channel. The overarching goal is to explore if a component for Named Entity Recognition could be a beneficial contribution to the Open Source project ‘Teledash’, to make NER available as a tool for journalists, NGOs, and researchers investigating the COVID-19 related protest movements and networks in Germany.

- approach of NER in German-language Telegram messages
- extraction of entities particularly relevant for mobilization / monitoring of potential targeting: PERSON, ORGANIZATION; DATE; TIME; LOCATION; ACTION as custom type
- explore potentials and limitations of existing models
- explore potentials of translation approach
- explore potentials of transfer learning with Transformer models

1.3 Thesis Structure

- Theory
- Exploration
- Implementation
- Conclusion and Discussion: suggestions for future work

Chapter 2

Theory

The first part of this chapter places Named Entity Recognition in its larger context of Natural Language Processing (NLP) in general, and of Information Extraction (IE) as a specific NLP task in particular. It provides a basic definition of of NER, lines out its historical evolvement, and discusses the challenges related to this task, both in general and with regard to the peculiarities of German language and to text data from social media.

The second part elaborates different approaches for NER, from knowledge- and rule-based methods over classical machine learning-based to deep learning-based approaches including transformers, and briefly discusses the limitations and potentials of the respective approaches.

2.1 Natural Language Processing

The field of Natural Language Processing combines approaches from computer science, linguistics, and artificial intelligence. Its main focus is to build systems and computational techniques for the automated processing and analysis of human language (cf. Vajjala et al. 2020, p. xvii).

Since its evolvement in the 1950s, NLP has been primarily an area of interest for academic research, but lately, it is being used more and more for

real-world tasks and business applications. (cf. Vajjala et al. 2020, p. xvii)

Core NLP applications comprise the use for email platforms (e.g. for spam filters), voice-based assistants, modern search engines, and machine translation services. (cf. Vajjala et al. 2020, p. 5). Furthermore, NLP is used for analyzing web and social media data (e.g. customer reviews), for spelling and grammar correction tools, and for a variety of domains including health care or finance. (cf. Vajjala et al. 2020, pp. 5–6).

Typical NLP tasks are for example language modeling, text classification, text summarization, information extraction, information retrieval, machine translation, or topic modeling (cf. Vajjala et al. 2020, pp. 6–7).

2.2 Information Extraction

Information Extraction (IE) is one of the several NLP tasks mentioned above. The aim of IE is to extract information from unstructured or semistructured natural language texts (cf. Zong, Xia, and Zhang 2021, p. 227).

‘Information’ can refer to different things, depending on the domain and purpose of the task: The extracted instances can be (named) entities such as people, locations, places, or organizations, but also important events occurring in texts (cf. Vajjala et al. 2020, p. 161). Sometimes the focus is on temporal information such as time and date, or on domain-specific entities, e.g. biomedical instances like genes, proteins, or viruses.

The extracted parts can also be attributes related to those entities, or relationships between the detected entities. The kinds of texts which information is extracted from may be news texts from the web, social media posts, customer reviews, academic literature, and others. The aim of IE is to generate structured output from this unstructured or semistructured text data. Typical tasks of IE are named entity recognition, entity disambiguation, relation extraction, and event extraction (cf. (Zong, Xia, and Zhang 2021, p. 227)

According to Zong, Xia, and Zhang 2021, IE technology can be classified via the domain aspect (limited vs. open domain), the type of extracted information (entity recognition, relation extraction, event extraction), and implementation methods (rule-based, machine learning-based, or deep learning-based approaches) (cf. Zong, Xia, and Zhang 2021, p. 229).

Common examples of IE applications are news tracking, counter-terrorism, business and financial intelligence, the processing of biomedical data or scientific libraries. IE can also be utilized for applications building on entity-oriented search, for question-answering systems, or the task of text segmentation (cf. Aggarwal 2018).

The origins of IE dates back to the late 1970s, and its development was further accelerated in the late 1980s, when the US government financed activities to evaluate IE technology. Organized by the *Defense Advanced Research Projects Agency* (DARPA), the first *Message Understanding Conference* (MUC-1) took place in 1987. The conference provided standard datasets for international researchers to compete on and was held seven times between 1987 and 1997. The tasks included NER, coreference resolution, relation extraction, and slot filling, and mainly focused on text data from limited domains such as naval military intelligence, terrorist attacks, or aircraft crashes (cf. Zong, Xia, and Zhang 2021, p. 228). The first NER task was introduced at the Sixth Message Understanding Conference (MUC-6) in 1996. The aim was to apply information extraction technologies for largely domain independent named entity recognition. Besides recognizing the entity types person, organization, and location, the task also involved numerical expressions including time, currency, and percentage (cf. Grishman and Sundheim 1996). The corpus for the task consisted of 318 annotated articles from the Wall Street Journal. For the MUC-7 NER task, Chinchor developed annotation guidelines which were used as a basis for several later NER tasks (cf. Chinchor 1997).

By the end of the 1990s, the MUC was replaced by the *Automatic Con-*

tent Extraction (ACE) meeting which put its focus on extracting more fine-grained entities, as well as entity relations, and events. There was also a shift in the data to broader news data covering political and international events.

In 2009, ACE became a subtask of the *Text Analysis Conference Knowledge Base Population* (TAC KBP). This shifted the focus in data to open domain data (mainly from webpages), and the focus of tasks to entity attribute extraction and entity linking. Both MUC and ACE contributed to the development of several standard test datasets for the evaluation and development of IE approaches. (cf. Zong, Xia, and Zhang 2021, p. 229).

2.3 Named Entity Recognition

Named entity recognition is a subtask of IE and addresses the task of identifying named entities like person, location, organization, drug, time, clinical procedure, biological protein, etc. in text. NER is often used as the first step in question answering, information retrieval, co-reference resolution, topic modeling, etc. and is thus a fundamental task in NLP (cf. Yadav and Bethard 2019).

NER can be divided into the following two subtasks (cf. Zong, Xia, and Zhang 2021, p. 229):

- **1. Entity detection**, to detect whether a given string in a text document is an entity, including the detection of the entity’s boundaries.
- **2. Entity classification**, to assign a specific category to the detected entity.

2.3.1 Common entity types

The entities which are to be identified via NER approaches are commonly represented by the following categories:

- PERSON
- LOCATION
- ORGANIZATION
- MISC/OTHER

Early NER approaches also included time, date, currency/quantity, and percentage as entity types. But while these can commonly be identified via regular expressions, instances of persons, locations, organizations, and miscellaneous/other usually do not follow a consistent pattern and thus pose larger challenges for NER, which is why most of NER research focusses mainly on these entity types (cf. Zong, Xia, and Zhang 2021, p. 229).

Also it should be noted that times, dates, currencies, quantities and percentage do not qualify as named entities in a narrow sense, but may be useful depending on the purpose of the application which NER is used for (cf. Aggarwal 2018, p. 386).

2.3.2 IOB tagging scheme

Since NER is usually regarded as a sequence labeling task, an appropriate label set as well as the language granularity for the annotation process needs to be defined for the respective sequence labeling model.

A widely used tagging scheme is the IOB (or BIO) label set. In this tagging scheme, the B-tag is used to label the beginning of an entity, the I-tag is used for labeling intermediate or final tokens of an entity, and the O-tag is assigned to all tokens outside of an entity.(cf. Zong, Xia, and Zhang 2021, p. 231).

Instances of different entity types representing persons, locations, or organizationa are thus labeled with two different tags per entity type: B-PER,

I-PER for persons, B-LOC, I-LOC for locations, and B-ORG, I-ORG for organizations (cf. Zong, Xia, and Zhang 2021, pp. 231–232). The usual granularity levels for the annotation of entities are usually word- and character-level (cf. Zong, Xia, and Zhang 2021, p. 232).

2.3.3 Evaluation of NER systems

Similar to other NLP tasks, NER involves an evaluation of the trained model. Evaluation can be done during model building, deployment, as well as in production, and rely on different metrics depending on the task at hand. Usually, the evaluation focuses on the model’s performance on unseen data.

The evaluation during model building and deployment usually relies on intrinsic evaluation using ML metrics, while the evaluation in the production phase also involves application-specific business metrics, which is called extrinsic evaluation.

Extrinsic evaluation is usually way more expensive than intrinsic evaluation, and often requires the involvement of stakeholders, sometimes even end-users. **Intrinsic evaluation** using machine learning metrics on the other hand can be done by machine learning engineers themselves and is less cost-effective. Also, they are applied in the earlier phase of the process, thus allowing early adjustments and quick improvements of the model (cf. Vajjala et al. 2020, p. 71). This is why intrinsic evaluation is usually used as a ‘proxy for extrinsic evaluation.’ ([71]Vajjala et al. 2020).

To measure a model’s performance by intrinsic evaluation, usually a test set is selected before training the model, and excluded from the training process. After training the model on the training set, the test set is used to evaluate the predictions of the model to assess its performance on unseen data. For NER, the manually annotated entity tags in the test set serve as gold reference with which the models predictions are compared (cf. Zong, Xia, and Zhang 2021, p. 241).

While intrinsic evaluation is a standard measure on most ML tasks, the

specific metrics applied differ depending on the specific task.

F1, Precision, and Recall

For NER models, the commonly used metrics are the model’s **precision**, **recall**, and **F1** score.

These metrics rely on the number of entities recognized correctly by the model (true positives, TP), the number of entities detected by the model, but not labeled as entities in the test set (false positives, FP), and the number of entities not recognized by the model, but tagged as named entities in the test set (false negatives, FN).

Precision, recall, and F1 are defined as follows:

$$precision = \frac{TP}{TP + FP} * 100$$

$$recall = \frac{TP}{TP + FN} * 100$$

$$F1 = \frac{2 * precision * recall}{precision + recall}$$

Precision thus represents the percentage of correctly identified named entities out of all recognized named entities, (i.e. the percentage of true positives out of all positive predictions), and recall equates to the percentage of named entities that are recognized by the model out of all tokens tagged as named entities in the test set.

The F1-score combines precision and recall and is the metric commonly used in shared tasks on NER to assess the performance of the participating systems.

2.3.4 Challenges for NER

Like other NLP tasks, NER is confronted with several challenges which stem from the characteristics of human language:

Ambiguity, understood as uncertainty or inexactness of meaning, is inherent to most human languages: Words or phrases can have multiple meanings, and usually it requires knowledge of the context in which they appear to resolve this ambiguity. Making computer-based systems take into account and ‘understand’ these contexts is one of the major challenges for NLP (cf. Vajjala et al. 2020, pp. 12–13). As for NLP tasks in general, ambiguity is also a core challenge for NER (cf. Aggarwal 2018, p. 386). For example, the word ‘Bill’ might refer to a person’s first name and thus indicate a named entity of type PERSON, but when located at the beginning of the sentence, it might also be a synonym for ‘invoice’.

For resolving this ambiguity, the **context** of a given word is usually of crucial relevance (cf. Aggarwal 2018, p. 387). NER systems differ in terms of how they can or cannot take into account the context of a word. Moreover, the data itself might not offer enough context. While articles from newspapers usually are of sufficient length and contain sufficient contextual information, social media texts are often of shorter length which results in a lack of context.

Common knowledge is often an essential part in a conversation between humans: Facts are assumed to be known and thus not expressed explicitly in statements. Humans constantly use common knowledge to understand language. One of the challenges for NLP is thus how to encode this common knowledge for computational approaches (cf. Vajjala et al. 2020, pp. 13–14).

Creativity is another feature inherent to human language: Even though language is constituted via and essentially shaped by rules, it is far from being exclusively rule-driven. Rather, language is creative and heterogeneous; characterized by various styles, genres, and dialects. One important challenge for NLP is thus to enable machines to adequately handle this creativity (cf. Vajjala et al. 2020, p. 14). Moreover, language changes and evolves over time, in interaction with social and cultural processes, as well as with the development of communication technology (one example being

the emergence of hashtags in the context of social media). NLP applications developed using data from the past might thus not fit well for data consisting of newly evolving manifestations of human language. These creative and dynamic characteristics of human language also pose challenges for the task of named entity recognition: Since new entities evolve over time, the set of known entities is never constant. While for ‘some types of entities like locations, the names of entities evolve slowly, whereas for others like people and organizations, the incompleteness problem is very significant.’ (Aggarwal 2018, p. 386).

Abbreviations referring to multiple instances of the same named entity are another important challenge (cf. Aggarwal 2018, p. 387). For example, the terms ‘U.S.’, ‘USA’ or ‘United States of America’ all refer to the same geopolitical entity. To overcome this challenge, usually entity disambiguation is an important step following named entity recognition.

Yet another challenge stems from **capitalization** conventions which vary across languages: While in English-language texts, capitalization usually indicates proper names and can thus be used as a feature for named entity recognition, in German-language texts all nouns are capitalized, making the feature less suitable (cf. Benikova, Biemann, and Reznicek 2014).

Diversity across languages makes it hard to apply an NLP solution from one language to another. According to the database *Ethnologue*, there are currently over 7.000 languages actively spoken worldwide (cf. Ethnologue 2022). Even though some of these languages share the same family and thus have some similarities, a direct mapping between two languages is not possible. Building language-agnostic solutions is a very complex matter, while building separate solutions for each language is labor- and time-intensive (cf. Vajjala et al. 2020, p. 14).

Imbalanced resources for different languages is yet another challenge for NLP, both in terms of research efforts and data. In fact, most NLP technologies are still designed for English or other European languages (cf.

Singh 2008). For these so-called high-resource languages, a large number of datasets and libraries exists while low-resource languages are usually significantly ‘less studied, resource scarce, less computerized’ (Magueresse, Carles, and Heetderks 2020). This imbalance is also reflected in the existing NER datasets.

2.3.5 Specific Challenges for NER in User-Generated Texts

The aforementioned challenges for NER also apply for texts from social media platforms. However, named entity recognition for these user-generated texts can be regarded as even more challenging.

One reason for this is the challenge of **domain adaptation**: Most of the existing NER systems are trained on a handful of standard NER datasets which mainly consist of newswire texts. Since social media text data is often **more informal and noisy** (e.g. due to misspellings) than newspaper articles, which aggravates domain adaptation for this genre and usually results in poor performance of existing NER systems when applied to texts from social media (cf. Ritter et al. 2011).

Figure 2.1 illustrates the noisy character of tweets as an example.

1	The Hobbit has FINALLY started filming! I cannot wait!
2	Yess! Yess! Its official Nintendo announced today that they Will release the Nintendo 3DS in north America march 27 for \$250
3	Government confirms blast n nuclear plants n japan...don't knw wht s gona happen nw...

Figure 2.1: Examples of noisy text in tweets, taken from Ritter et al. 2011

Furthermore, since texts from social media are often **shorter of length**, they lack sufficient context for the determination of an entity's type (cf. Ritter et al. 2011).

The **wide variety of capitalization styles** are another challenge, like e.g. the use of only lowercase for German texts, or the use of uppercase only in some texts (cf. Ritter et al. 2011). As Ritter et al. 2011 point out, news-trained NER systems seem to rely heavily on capitalization which limits their applicability to user-generated social media texts.

Also, these texts usually contain more **out of vocabulary (OOV) words** than e.g. news articles. This is partly due to misspellings, but also stems from platform specific codes and styles: The frequent occurrence of **special characters** such as # for hashtags or @ for user mentions poses challenges because they cannot easily be handled during annotation and training. User-generated text in social media also often includes a lot of **lexical variations**. As an example, Ritter et al. 2011 collected more than fifty variations for the word 'tomorrow' from a Twitter corpus, as shown in figure 2.2.

‘2m’, ‘2ma’, ‘2mar’, ‘2mara’, ‘2maro’,
 ‘2marrow’, ‘2mor’, ‘2mora’, ‘2moro’, ‘2mo-
 row’, ‘2morr’, ‘2morro’, ‘2morrow’, ‘2moz’,
 ‘2mr’, ‘2mro’, ‘2mrrw’, ‘2mrw’, ‘2mw’,
 ‘tmmrw’, ‘tmo’, ‘tmoro’, ‘tmorrow’, ‘tmoz’,
 ‘tmr’, ‘tmro’, ‘tmrow’, ‘tmrrow’, ‘tm-
 rrw’, ‘tmrw’, ‘tmrww’, ‘tmw’, ‘tomaro’,
 ‘tomarow’, ‘tomarro’, ‘tomarrow’, ‘tomm’,
 ‘tommarow’, ‘tommarrow’, ‘tommoro’, ‘tom-
 morow’, ‘tommorrow’, ‘tommorw’, ‘tomm-
 row’, ‘tomo’, ‘tomolo’, ‘tomoro’, ‘tomorow’,
 ‘tomorro’, ‘tomorrrw’, ‘tomoz’, ‘tomrw’,
 ‘tomz’

Figure 2.2: Lexical variations of the word ‘tomorrow’ in tweets, taken from Ritter et al. 2011

Last but not least, social media texts often **mixed-language texts**, making it harder for language-specific models to accurately determine named entities in them.

Named entity recognition applications for other domains thus usually require the creation of custom corpora including a labor-intensive annotation process, or rely on automatically generated annotations of large text corpora, often resulting in inconsistent annotations.

2.3.6 Important Shared Tasks and Datasets for German-Language NER

The NER tasks at the Conference on Computational Natural Language Learning (ConLL) for English and German in 2003 was an important cornerstone for German-language NER (cf. Tjong Kim Sang and De Meulder

2003)

The provided German dataset consists of 909 articles sampled from the newspaper *Frankfurter Rundschau* (cf. Tjong Kim Sang and De Meulder 2003). The task focused on the entity types person (PER), location (LOC), organization (ORG), and miscellaneous (MISC).

The ConLL 2003 dataset is publicly accessible, but no licensing information is available. Furthermore, given the fact that the corpus only contains newspaper articles, its applicability to other domains might be limited. The annotation of the dataset was carried out at the University of Antwerp (cf. Tjong Kim Sang and De Meulder 2003), thus most likely by non-native speakers, which might result in inconsistencies of the labeled data.

In 2014, GermEval, a series of shared tasks focussing on Natural Language Processing for German-language, announced a task for Named Entity Recognition for German-language texts (cf. Benikova, Biemann, and Reznicek 2014).

The provided dataset was compiled from German Wikipedia articles and online news. Similar to the ConLL 2003 task, entity types of interest were person (PER), organization (ORG), location (LOC), and other (OTH).

The dataset is publicly available under a e CC-BY license. However, since the GermEval2014 dataset also includes nested and derived entities, it is not readily combinable with other datasets. And even though the corpus was created from online resources, it does not contain noisy user-generated texts, limiting its applicability to other social media texts.

Up until today, the Germeval 2014 corpus and (to some extent) the ConLL 2003 dataset represent standards for German-language NER and are used to train and evaluate state-of-the art NER models.

2.3.7 Important Experimental Studies on NER on Social Media Texts

In 2011, Ritter et al. conducted an experimental study on named entity recognition in tweets, and used a random sample of 2.400 English-language tweets for this which were annotated with the following ten entity types: PERSON, GEO-LOCATION, COMPANY, PRODUCT, FACILITY, TV-SHOW, MOVIE, SPORTSTEAM, BAND, and OTHER (cf. Ritter et al. 2011). The best participating system in this task achieved an F1 score of 0.66 for these ten types (cf. Ritter et al. 2011).

A more detailed look into the results for the standard entity types PERSON, LOCATION, and ORGANIZATION, reveals that the F1 score for the entity type PERSON ranks highest with up to 0.83, followed by LOCATION, and then ORGANIZATION (0.74 and 0.66) for the best participating system.

In 2015, Derczynski et al. investigated the applicability of existing, by then state-of-the-art systems for named entity recognition in tweets. Their results indicate as well that existing machine-learning based approaches do not perform well on noisy user-generated data from microblogs, using three different English-language datasets, including the corpus created by Ritter et al. 2011.

The evaluated systems achieved F1 scores ranging between 40.27, 51.49 (Ritter dataset), and 77.18 (cf. Derczynski et al. 2015).

In line with Ritter et al. 2011, the authors identify unreliable capitalization, typographic errors, shortenings, and lack of context due to the shortness of the texts as main challenges in this specific domain.

The authors suggest language detection, part-of-speech tagging, and normalization as pre-processing steps to encounter these challenges. Their results indicate however that e.g. normalization only leads to minimal increases, and in some cases even decreases the F1 score (cf. Derczynski et al. 2015).

2.3.8 Important Shared Tasks and Datasets for NER on Social Media Texts

Also in 2015, Baldwin et al. organized a shared task for Named Entity Recognition on social media data, namely tweets from Twitter in the context of the *Workshop on Noisy User-generated Text* (W-NUT). The task used the corpus created by Ritter et al. 2011.

Among the participating systems, the best F1 score achieved was 56.41, which is ~ 0.1 lower than the F1 score achieved in the initial study by Ritter et al. 2011, and ~ 0.2 lower than the best F1 score achieved at the GermEval 2014 NER task (76.38, cf. Benikova et al. 2014).

The W-NUT series continued to address the task of NER in subsequent years: In 2016, another shared task on Twitter Named Entity Recognition was organized in the context of the workshop, again relying on the Ritter dataset and focussing on the ten entity types included in it. An important difference compared to 2015 was that some of the participating systems introduced the use of neural network methods for the task of NER (cf. Strauss et al. 2016), including the winning system which achieved an F1 score of 59.08.

The W-NUT shared task in 2017 focused on novel and emerging entity recognition, thus addressing the challenge of newly emerging entities in social media data. The Ritter corpus was used as training data again. The development set consisting of ~ 1.000 documents was composed of manually annotated texts from different online sources such as Reddit, StackExchange, and Twitter, while the test set (~ 1.300 documents) consisted of manually annotated Youtube comments (cf. Derczynski et al. 2017). The annotation schema mapped the ten entity types of the Ritter corpus to the seven entity types person, location (including GPE, facility), corporation, product, creative-work, and group (representing music bands, sports teams, and non-corporate organisations). The best participating system achieved an F1 score of 41.86. In the following years, the focus of the workshop series shifted to

other tasks and domains which might be due to newly emerging research trends, but also due to the lack of relevant progress in terms of performance. Judging from the F1 scores of the respective winning systems, the task of NER in social media texts continued to be a considerable challenge. Furthermore, the ongoing shortage of manually annotated and sufficiently large corpora for this task becomes apparent. However, the results might also indicate that by then state-of-the-art machine learning systems had reached their limits. The following section traces the evolvement of NER approaches over time and gives insights into the respective potentials and limitations.

2.4 Overview of NER approaches

Since its beginnings in the 1970s, NER approaches have undergone several phases of development, parallel to the advancements in NLP technologies: Starting with early rule-based and knowledge-based systems, the field moved on to machine-learning approaches based on feature-engineering. The development of NER systems based on neural networks, relying only on minimal feature-engineering, marked another important step forward in the field. Lately, the introduction of transformer models brought along yet another crucial advancement and achieve new state of the art performance for several NLP tasks including NER.

2.4.1 Rule-based and Knowledge-based Approaches

The first systems for NER relied on handcrafted rules, lexicons, orthographic features and ontologies (cf. Yadav and Bethard 2019).

Rule-based systems make use of the fact that the structure and the context of named instances of type person, location, and organization often follow certain rules and can thus be recognized to some extent by using e.g. regular expressions (cf. Zong, Xia, and Zhang 2021, p. 230). Hand-crafted rules can make use of features such as capitalization (used in a lot of lan-

guages for the beginning of person names) or of certain ‘key words’ such as titles for persons (Mr., Mrs. Dr., etc.) for the recognition of named entities (cf. Zong, Xia, and Zhang 2021, p. 230).

However, these approaches are limited even when a large amount of data is considered. This is due to the ambiguity of language, resulting in different meanings for a given word or sequence in different contexts. Also, these approaches are rather static and often struggle with handling newly emerging identities without updating the underlying rules. Furthermore, the generation of rules is labor-intensive and usually requires expert domain knowledge. (cf. Zong, Xia, and Zhang 2021, p. 231).

Knowledge-based approaches rely on lexicon resources and domain specific knowledge. The performance of these systems heavily depends on the exhaustiveness and completeness of the used lexicon. Here as well, the process can be rather labor-intensive because domain expert knowledge is needed for the construction and maintenance of the knowledge resources (cf. Yadav and Bethard 2019).

2.4.2 Machine-learning Based Approaches

Machine-learning based approaches based on feature-engineering introduced a critical advancement for the task of NER (cf. Nadeau and Sekine 2007)

Machine learning approaches resign from the concept of handcrafted rules and instead focus on the development of algorithms which are able to perform a given task by automatically learning from a large number of examples, the training data (cf. Vajjala et al. 2020, p. 14). The process usually involves the creation of a numeric representation of the training data. These features are then processed by the model which is supposed to patterns in the training data from them (cf. Vajjala et al. 2020, p. 15).

The introduction of machine learning approaches thus allowed for the development of NER models through an automated learning process.

Machine learning algorithms can roughly be grouped into three paradigms:

supervised learning, unsupervised learning, and reinforcement learning. For the sake of brevity, I will focus on supervised learning and not go into further detail about the latter two concepts.

Feature-engineered supervised systems

In supervised learning, the machine learning model learns to make predictions for unseen data from a large number of examples which represent input-output pairs. Regarding NER, an input example could be a text containing different instances of entities. The output would then be the corresponding spans and labels which determine the type of each given entity in the text example. These outputs are also called labels or ground truth. In order to yield a well-performing model which is able to predict the spans and labels for entities in unseen texts, large amounts of labeled data are required (cf. Vajjala et al. 2020, p. 15).

Commonly used models for NER based on machine learning are Hidden Markov Models (HMM), Maximum Entropy Models (MEM), Support Vector Machines (SVM), Decision Trees, and Conditional Random Fields (CRF) (cf. Yadav and Bethard 2019)

Support Vector Machines - goal of any classification task: learn a decision boundary that acts as a separation between different categories of text - decision boundary can be linear or nonlinear (e.g. a circle) - SVM: can learn linear and nonlinear - SVM learns an optimal decision boundary so that the distance between points across classes is at its maximum Vajjala et al. 2020, p. 20

Hidden Markov Model HMM is a statistical model that assumes there is an underlying, observable process with hidden states that generates the data, ie we can only observe the data once it is generated - an HMM then tries to model the hidden states from this data - e.g. POS tagging: we

assume that text is generated according to an underlying grammar, which is hidden underneath the text - hidden states are parts of speech that inherently define the structure of the sentence following the language grammar, but we only observe the words that are governed by these latent states - HMM also make the Markow assumption: each hidden state is dependent on the previous state(s) - human language is sequential which means that the current word depends on what occurred before - this makes HMM a powerful tool for modeling textual data Vajjala et al. 2020, pp. 21–22

Conditional Random Field - another algorithm used for sequential data - performs a classification task on each element in the sequence - takes the sequential input and the context of tags into consideration Vajjala et al. 2020, p. 22

Limitations: feature-engineering (aufwändig) sparse vectors no word embeddings what about OOV words?

2.4.3 The Introduction of Neural Networks

Deep learning: "branch of machine learning that is based on artificial neural network architectures." (cf. Vajjala et al. 2020, p. 14)

NER systems have been studied and developed widely for decades but accurate systems using deep neural networks (NN) have only been introduced in the last few years (Yadav and Bethard 2019)

Starting with Collobert et al. (2011), neural network NER systems with minimal feature engineering have become popular

are appealing because they typically do not require domain specific resources like lexicons or ontologies, and are thus poised to be more domain independent. Various neural architectures have been proposed, mostly based on some form of recurrent neural networks (RNN) over characters, sub-words and/or word embeddings. Yadav and Bethard 2019

Feature-inferring neural network systems Collobert and Weston (2008)

proposed one of the first neural network architectures for NER, with feature vectors constructed from orthographic features (e.g., capitalization of the first character), dictionaries and lexicons. Yadav and Bethard 2019

Later work replaced these manually constructed feature vectors with word embeddings (Collobert et al., 2011), which are representations of words in n -dimensional space, typically learned over large collections of unlabeled data through an unsupervised process such as the skip-gram model (Mikolov et al., 2013). Studies have shown the importance of such pre-trained word embeddings for neural network based NER systems (Habibi et al., 2017), and similarly for pre-trained character embeddings in character-based languages like Chinese Modern neural architectures for NER can be broadly classified into categories depending upon their representation of the words in a sentence. For example, representations may be based on words, characters, other sub-word units or any combination of these Yadav and Bethard 2019

Feedforward neural networks

2.4.4 Recurrent Neural Networks

Recurrent neural networks - language is inherently sequential - thus, a model which can progressively read an input text from one end to another can be very useful for language understanding - RNNs are designed to keep such a sequential processing and learning in mind - have neural units that are capable of remembering what they have processed so far - memory is temporal, information is stored and updated with every step as the RNN reads the next word as input Vajjala et al. 2020, p. 23

Long short-term memory - RNNs cannot remember longer contexts - do not perform well when input text is long - LSTM = solve this issue - concept: forget irrelevant context, only remember relevant context for task

at hand - relieves the load of having to remember a lot of context for long inputs Vajjala et al. 2020, p. 23

Bi-LSTM

Convolutional neural networks - used heavily in computer vision tasks (e.g. image classification) - also used for NLP - each word in a sentence is represented as a word vector, all word vectors of sentence have the same size - resulting matrix representing the sentence can now be treated similar to an image by the CNN - advantage: CNN can look at a group of words together using a context window - CNN uses a collection of convolution and pooling layers to achieve a condensed representation of the text which is then fed as input to a fully connected layer to learn some NLP tasks like text classification Vajjala et al. 2020, p. 24

2.4.5 Transformers

- latest entry in the league of deep learning models for NLP - have achieved state of the art in almost all major NLP tasks in recent years - model the textual context but not in a sequential manner - given a word in the input, it prefers to look at all the words around it (known as self-attention) and represent each word with respect to its context - e.g. bank different meaning depending on context (financial institute, furniture) - transformers can model such context and hence have been used heavily in NLP tasks due to this higher representation capacity as compared to other deep networks Vajjala et al. 2020, pp. 25–26

- prior to transformers: recurrent architectures (e.g. LSTMs, RNNs) - recurrent architectures contain a feedback loop in the network connections; allows information to propagate from one step to another, making them ideal for modeling sequential data like text - RNN: receives input, feeds it through the network, outputs a vector called the hidden state - at the same time,

the model feeds some information back to itself through the feedback loop, which it can then use in the next step - RNN passes information about its state at each step to the next operation in the sequence - this allows an RNN to keep track of information from previous steps, and use it for its output predictions Tunstall et al. 2022, p. 2

Transfer learning for downstream tasks with transformers - large transformers used for smaller downstream tasks via transfer learning - transfer learning = technique where the knowledge gained while solving one problem is applied to a different but related problem - transformers: pre-training (training of very large transformer models, unsupervised) to predict a part of a sentence given the rest of the content so it can encode the high-level nuances of the language in it - models are trained on more than 40GB of textual data, scraped from the whole internet - this model is then fine-tuned on NLP downstream tasks, e.g. NER Vajjala et al. 2020, p. 26

2.5 Summarization

Chapter 3

Qualitative Exploration of Existing Solutions

In order to explore the potentials and limitations of existing German-language models for Named Entity Recognition, two models were selected for a basic qualitative exploration of their results with some sample texts.

While a quantitative evaluation would be the ideal approach, this is not possible since there are no existing (German-language) models which support all the entity types relevant for the NER task as defined here. Precisely, most of the models are limited to the four common entity types PERSON, LOCATION, ORGANIZATION, and OTHER/MISC. To the best of my knowledge, there are no existing models supporting the detection of the entity types TIME, DATE, and ACTION. An exception is the English-language spaCy model which supports TIME and DATE recognition. Against this background, it will explore the potentials of machine translation to allow for the use of well-resourced English NER models for the recognition of named entities in the selected example texts.

3.1 German-language Named Entity Recognition with spaCy and Flair

The models explored in this chapter were selected from the library spaCy and the framework FLAIR. The models were chosen due to their easy applicability, allowing for fast exploration and analysis, and because both of them support German- and English-language named entity recognition.

3.1.1 The German-language spaCy model

spaCy is a free open-source library for Natural Language Processing developed for use in production. The library is extensively documented and thus serves as an easy starting point for beginner, while also claiming to be designed specifically for use in production. The library supports more than 66 languages (including German), comes with pre-trained word vectors, and supports a variety of NLP tasks such as part-of-speech tagging, text classification, and named entity recognition (cf. ExplosionAI 2022c). The German spaCy model supports the common entity types are LOC, MISC, ORG, and PER (cf. ExplosionAI 2022d).

The central component of the model’s NER pipeline is spaCy’s Entity Recognizer which ‘identifies non-overlapping labelled spans of tokens’ (ExplosionAI 2022b) and uses a transition-based algorithm which assumes that the most relevant information about an entity is close to its first token (cf. ExplosionAI 2022b). The underlying model is a neural network state prediction model, spaCy’s TransitionBasedParser. Transition-based parsing can be described as ‘an approach to structured prediction where the task of predicting the structure is mapped to a series of state transitions.’ (ExplosionAI 2022e). The model is composed of two to three subnetworks which take care of token-vectorization, feature-specific vectorization for each token-feature pair and then computing the state representation from them, and (optionally) a subnetwork which predicts scores from the state representation (oth-

erwise the outputs from the second subnetwork will be used for prediction) (cf. ExplosionAI 2022e).

The model was trained on the WIKINER corpus, a free, multilingual corpus created from Wikipedia articles which were automatically annotated and thus are silver standard training data (cf. Nothman et al. 2013). According to the official documentation, the model achieved an F1 score of 0.84 (cf. ExplosionAI 2022d).

3.1.2 The German-language standard NER Flair model

The FLAIR framework is an NLP library released in 2018, developed to ‘facilitate training and distribution of state-of-the-art sequence labeling, text classification and language models’ (Akbik et al. 2019). The framework’s aim is to provide an interface to easily use different types of word and document embeddings (cf. Akbik et al. 2019).

The German-language FLAIR model for NER uses a combination of GloVe embeddings (cf. Pennington, Socher, and Manning 2014 and Flair embeddings to create the word embeddings. The conceptual approach behind the Flair embeddings is described in detail in Akbik, Blythe, and Vollgraf 2018. It can briefly be characterized as an approach which creates ‘contextual string embeddings’ (Akbik, Blythe, and Vollgraf 2018) which take into account the context of a given text, resulting in different embeddings for the same word depending on its context in a document (cf. Akbik, Blythe, and Vollgraf 2018). The word embeddings passed to a BiLSTM-CRF sequence labeling model which takes care of the final downstream sequence labeling task (cf. Akbik, Blythe, and Vollgraf 2018).

The German Flair model used here is the German standard 4-class NER model by Flair. The model was trained on the ConLL 2003 corpus, and reached an F1 score of 87.94 according to the model’s HuggingFace documentation (cf. Akbik, Blythe, and Vollgraf 2022b).

3.1.3 Example Data

For the exploration of the two models, ten example messages were randomly selected from the Telegram channel ‘Demotermine’ (*Demo dates*).

The channel was selected because a prior qualitative exploration of sample messages indicated that its content consists mainly of information about and mobilization for demonstrations and others forms of protests against the German COVID-19 containment measures and could thus be expected to contain a lot of relevant entities.

3.1.4 Implementation

For exploring the German medium-size spaCy model, a simple code was implemented to predict and print the detected entities:

```
import spacy
nlp = spacy.load("de_core_news_md")

def get_entities(texts):
    for text in texts:
        doc = nlp(text)
        if doc.ents:
            print("Text: ", text, '\n')
            print("The following NER tags are found: ",
                  [(ent.text, ent.label_) for ent in doc.ents],
                  '\n\n')
get_entities(texts)
```

3.1. GERMAN-LANGUAGE NAMED ENTITY RECOGNITION WITH SPACY AND FLAIR32

The next code section shows the implementation for the Flair model:

```
from flair.models import SequenceTagger
from flair.data import Sentence

tagger = SequenceTagger.load("flair/ner-german")

def get_entities(texts):
    for text in texts:
        # transform example text to Sentence
        sentence = Sentence(text)
        # use tagger to predict NER tags in sentence
        tagger.predict(sentence)
        print(sentence, '\n')
        print('The following NER tags are found:')
        for entity in sentence.get_spans('ner'):
            print(entity)
        print('\n\n')
get_entities(texts)
```

3.1.5 Results

Table 3.1 shows the results of both models for the example texts:

I would like to emphasize that due to the small size of the explored sample, these explorations are rather anecdotal and cannot be generalized without further quantitative evaluation on a larger sample.

However, the following observations can be made when looking at the results:

The spaCy model produces more false positives overall: It falsely tags the sequences ‘Endkundgebung’ (final rally), ‘GEHT DEMONSTRIEREN’ (go demonstrate), ‘Dresdner &’, ‘Videos’, ‘Berliner’, ‘Versammlungsfre’ (freedom of assembly, truncated), ‘Einfach’ (simple/simply), ‘Der Mann mit dem Schild’ (the man with the sign), and ‘Einsatzwägen’ (patrol cars) as entities.

The Flair model only falsely tags ‘FU’ and ‘Demo-Berlin’. The sequence

3.1. GERMAN-LANGUAGE NAMED ENTITY RECOGNITION WITH SPACY AND FLAIR33

Table 3.1: Detected entities for German-language example messages

No.	Text	spaCy/de_core_news_md	flair/ner-german
1	Freiburg Aufzug mit Endkundgebung am Platz der alten Synagoge . Live Musik ! Details 19062021 Raus auf die Straße	Endkundgebung[LOC], alten Synagoge[LOC]	Freiburg[LOC]
2	GEHT DEMONSTRIEREN ! - Message von Elijah Tee an die Dresdner & anderen Demonstranten in Deutschland Morgen am 13.06. um 16.30 Uhr und bundesweit überall , wo es Mitdenker gibt . - Bilder und Videos hochladen - Diskussion Fahren oder Mitfahren	GEHT DEMONSTRIEREN [ORG], Elijah Tee[PER], Dresdner &[LOC], Deutschland[LOC], Videos[MISC]	Elijah Tee[PER], Deutschland[LOC]
3	15831 Blankenfelde Mahlow Berliner Speckgürtel Ausfahrt Lichtenrade am 14.03.2021 14032021 Raus auf die Straße	Blankenfelde Mahlow[PER], Berliner[MISC]	Blankenfelde Mahlow[LOC], Lichtenrade[LOC]
4	2021 10 11 Sven Liebich fragt Ministerpräsident Haseloff nach dessen Verständnis von Versammlungsfre - Sven Liebich 0:23 Raus auf die Straßen Übersicht / Overview	Sven Liebich[PER], Haseloff[PER], Versammlungsfre[PER], Sven Liebich[PER]]	Sven Liebich[PER], Haseloff[PER], Sven Liebich[PER]
5	FÜR ALLE DEMOKRATEN GEFAHR : Die Polizei blockiert den Stern ! Deswegen laufen die Menschen vom Ernst-Reuther-Platz nach Südosten zum Breitscheidplatz und dort weiter auf den Kudamm ! ! Schließt euch an : Neue Kantstraße direkt Richtung Osten auf den Breitscheidplatz ! Von der Straße des 17. Juni Richtung Süden zum Kudamm !	DEMOKRATEN[ORG], Ernst-Reuther-Platz[LOC], Breitscheidplatz[LOC], Kudamm ! ! [LOC]), Neue Kantstraße[LOC], Breitscheidplatz[LOC], Kudamm ! [LOC]	FU[ORG], Ernst-Reuther-Platz[LOC], Breitscheidplatz[LOC], Kudamm[LOC], Kantstraße[LOC], Breitscheidplatz[LOC], Kudamm[LOC]
6	Dr. Gunter Frank : Wie Moralismus dazu führt das man Feinde sieht und vernichten will — Antihygienista - Offene Gesellschaft Kurpfalz 15:04 Raus auf die Straßen Übersicht / Overview	Gunter Frank[PER], Antihygienista[LOC], Offene Gesellschaft Kurpfalz[ORG]	Gunter Frank[PER], Antihygienista[ORG], Offene Gesellschaft Kurpfalz[ORG]
7	Einfach nur Krank solche Politiker ! Was sollen wir mit ihm machen wenn unsere Kinder anhand dieser Impfung nen schaden erleiden ? Wer ist dann dafür verantwortlich , wen schmeißen wir dann in den Knast und nehmen sein ganzes Geld weg , sollen wir das mit dir machen ? Raus auf die Straße	Einfach [MISC]	<i>no entities detected</i>
8	27.11.2021 Frankfurt Durchsagen von gemischt mit Musik . Super Idee Für mehr Berichte folgen auf ... Raus auf die Straßen Übersicht / Overview	Frankfurt[LOC]	Frankfurt[LOC]
9	Demo-Berlin : Der Mann mit dem Schild wurde soeben von der Polizei festgenommen . Circa 7 Einsatzwägen sind vor Ort und umzäunen den Park großräumig . Am Humboldthain gibt es nichts mehr zu gewinnen . # b2908 Raus auf die Straßen Übersicht / Overview	Der Mann mit dem Schild[LOC], Einsatzwägen[LOC], Humboldthain[LOC]	Demo-Berlin[LOC], Humboldthain[LOC]
10	Anthony Fauci : Definition von “ vollständig geimpft ” könnte geändert werden — Epoch Times Nachrichten 6:59 Raus auf die Straßen Übersicht / Overview	Anthony Fauci[PER], Times Nachrichten[MISC]	Anthony Fauci[PER], Epoch Times Nachrichten[ORG]

‘FU’ was originally part of the German word ‘FÜR’ and apparently was modified by the model’s tokenizer which is applied when creating a Sentence object from the input text. The sequence ‘Demo-Berlin’ contains a location, which makes the model’s mistake to some extent plausible.

It can also be observed that the spaCy model sometimes assigns the wrong tag to a correctly detected entity. This applies for the sequences ‘Blankenfelde Mahlow’ which is not a person as suggested by the model, but a location.

Regarding false negatives, it can be observed that the spaCy model does not detect ‘Freiburg’ in the first message. On the other hand, it recognizes parts of the sequence ‘Platz der alten Synagoge’ (square of the old synagogue) as a location. The Flair model correctly detects Freiburg, but misses the mentioned square.

Even though the sample is too small for generalization, this first exploration casts strong doubts on the claim that spaCy offers production-ready components for NER.

It can be hypothesized that both the different model architectures and the different corpora used for training result in the better performance of the Flair model compared to the spaCy model.

3.2 Exploring the Potentials of Machine Translation for NER

Even though the spaCy model explored in the previous section provided rather dissatisfying results, its English equivalent has one advantage, since it supports more entity types, including DATE and TIME, meaning that it might be useful for extracting dates and times, as they are relevant for the aim of this thesis. Also, since more training data is available for English-language NLP models, exploring the potentials of machine translation for NER seems worth a try.

3.2.1 The English-language spaCy model

The English-language spaCy model supports 18 entity types: (CARDINAL, DATE, EVENT, FAC, GPE, LANGUAGE, LAW, LOC, MONEY, NORP, ORDINAL, ORG, PERCENT, PERSON, PRODUCT, QUANTITY, TIME, and WORK_OF_ART).

Table 3.2 shows the definition for each entity type (obtained for each entity via the command `spacy.explain{’<ENTITY>’}`):

Table 3.2: Overview of spaCy’s entity types

Entity type	Definition
CARDINAL	Numerals that do not fall under another type.
DATE	Absolute or relative dates or periods.
EVENT	Named hurricanes, battles, wars, sports events, etc.
FAC	Buildings, airports, highways, bridges, etc.
GPE	Countries, cities, states.
LANGUAGE	Any named language.
LAW	Named documents made into laws.
LOC	Non-GPE locations, mountain ranges, bodies of water.
MONEY	Monetary values, including unit.
NORP	Nationalities or religious or political groups.
ORDINAL	”first”, ”second”, etc.
ORG	Companies, agencies, institutions, etc.
PERCENT	Percentage, including ”%”.
PERSON	People, including fictional.
PRODUCT	Objects, vehicles, foods, etc. (not services).
QUANTITY	Measurements, as of weight or distance.
TIME	Times smaller than a day
WORK_OF_ART	Titles of books, songs, etc.

The training data sources listed in the model’s official documentation suggest that the NER training data was the OntoNotes 5 corpus (cf. Ralph Weischedel 2013). The model achieved an F1 score of 0.85 for NER.

3.2.2 The English-language Flair model

The architecture of the standard English-language Flair model is the same as for the standard German model described in section 3.1.2. It was trained on a corrected version of the CoNLL 2003 corpus and achieved an F1 score of 93.06 (cf. Akbik, Blythe, and Vollgraf 2022a).

3.2.3 Implementation

To explore the potentials of the medium-size English spaCy model and the Flair English-language standard model, the example texts were translated into English using the DeepL API:

```
import deepl
from src.config import deepl_credentials
auth_key = deepl_credentials.auth_key

# initialize and authenticate translator
translator = deepl.Translator(auth_key)

# method to get translated texts
def translate(text, translator, target_lang, formality="less"
              ):
    result = translator.translate_text(text, target_lang=
                                      target_lang)
    return result.text

# apply translation to cleaned texts
df['text_EN'] = df.cleaned_text.apply(lambda x: translate(x,
                                                         translator, 'EN-US'))
```

The translated texts were then fed to the English-language spaCy and Flair model, using the same code as described in section 3.1.4.

3.2.4 Results

Table 3.3 shows the results of both models for the translated example texts.

3.3 Summarization

potentials and limitations / observed errors - spacy - Flair explorative evaluation

Table 3.3: Detected entities for English-language example messages

No.	Text	spaCy/en_core_web_md	flair/ner-english
1	Freiburg procession with final rally at the square of the old synagogue . Live music ! Details 19062021 Out on the street	<i>no entities detected</i>	Freiburg[LOC]
2	GO DEMONSTRATE ! - Message from Elijah Tee to the Dresdeners & other demonstrators in Germany Tomorrow on 13.06. at 16.30 clock and nationwide everywhere , where there are fellow thinkers . - Upload pictures and videos - discussion drive or ride along	Germany[GPE], Tomorrow[DATE], 13.06[CARDINAL], 16.30[CARDINAL]	Elijah Tee[PER], Dresdeners[MISC], Germany[LOC]
3	15831 Blankenfelde Mahlow Berlin Speckgürtel Lichtenrade exit on 03/14/2021 14032021 Out on the road	15831[DATE], Blankenfelde Mahlow Berlin[PERSON], Speckgürtel Lichtenrade[PERSON], 03/14/2021 14032021[DATE]	"Blankenfelde Mahlow Berlin Speckgürtel Lichtenrade"[MISC]
4	2021 10 11 Sven Liebich asks Prime Minister Haseloff about his understanding of assembly free - Sven Liebich 0:23 Raus auf die Straßen Overview / Overview	Sven Liebich[ORG], Haseloff[PER], Straßen Overview / Overview[ORG]	Sven Liebich[PER], Haseloff[PER], Sven Liebich[PER], Raus auf die Straßen Overview[ORG]
5	FOR ALL DEMOCRATS DANGER : The police blocks the Stern ! That's why people run from Ernst-Reuther-Platz to the southeast to Breitscheidplatz and there further to Kudamm ! ! Join : Neue Kantstraße directly to the east to Breitscheidplatz ! From the Straße des 17. Juni southwards to the Kudamm !	DEMOCRATS[NORP], Ernst-Reuther-Platz[ORG], Breitscheidplatz[PERSON], Kudamm[FAC], Neue Kantstraße[PERSON], Straße[GPE], 17[CARDINAL], Kudamm[FAC]	Stern ![MISC], Ernst-Reuther-Platz[LOC], Breitscheidplatz[LOC], Kudamm[LOC], Neue Kantstraße[ORG], Breitscheidplatz[LOC], Straße des[LOC], Juni[LOC], Kudamm[LOC]
6	Dr. Gunter Frank : How moralism leads to the fact that one sees enemies and wants to destroy - Antihygienista - Offene Gesellschaft Kurpfalz 15:04 Raus auf die Straßen Übersicht / Overview	Gunter Frank[PER], 15:04 Raus auf[PER], Straßen Übersicht / Overview[ORG]	Gunter Frank[PER]
7	"Just sick such politicians ! What should we do with him if our children suffer from this vaccination ? Who is responsible for it , who do we throw in jail and take away all his money , shall we do that with you ? Out on the street"	<i>no entities detected</i>	<i>no entities detected</i>
8	27.11.2021 Frankfurt announcements of mixed with music . Super idea For more reports follow on ... Out on the streets Overview	Frankfurt[GPE]	Frankfurt[LOC]
9	Demo-Berlin : The man with the sign has just been arrested by the police . About 7 emergency vehicles are on site and fence the park extensively . At the Humboldthain there is nothing more to win . # b2908 Out on the streets	Humboldthain[PER], #[CARDINAL]	Humboldthain[LOC]
10	Anthony Fauci : Definition of " fully vaccinated " could be changed - Epoch Times News 6:59 Out on the Streets Overview / Overview.	Anthony Fauci[PER], the Streets Overview / Overview[ORG]	Anthony Fauci[PER], Epoch Times News[ORG], Streets Overview[MISC]

Chapter 4

Towards building a custom model for NER in Telegram channels

4.1 Data

4.1.1 Existing German-language datasets for this specific task

GermEval 2014 NER task dataset

Why not GermEval? TIME, DATE missing; ACTION as well not social media data

DFKI Smartdata corpus

the DFKI SmartData Corpus is released as CC-BY 4.0.

XTREME dataset

PAN-X annotations were generated through an automated process - only ‘silver standard’, not used here

WIKIANN

?

4.1.2 Domain-specific data from Telegram channel

For the creation of a dataset suitable for the aim of this thesis, the Telegram channel ‘Demotermine’ (*Demo dates*) was selected.

The channel was selected because a prior qualitative exploration of sample messages indicated that its content consists mainly of information about and mobilization for demonstrations and others forms of protests against the German COVID-19 containment measures and could thus be expected to contain a lot of entities of the types relevant for the aim of this thesis. The focus of the channel is also reflected in the extended channel name ‘Demo-Termine & Kontakte, Gleichgesinnte treffen - Demokalender, Spaziergänge, Mahnwachen, Meditation *Rücktritt Bundesregierung!’ (*Demo dates & contacts, meet like-minded - Demo calendar, walks, vigils, meditation *resignation federal government!*).

The channel was part of a network investigated by IDZ Jena, and in the context of the research project ‘Digitaler Hass’, data from the channel was collected including messages from the start of the channel until December 19, 2021.

By the beginning of June 2022, the channel had around 52.900 subscribers.

The dates of the collected messages range between March 3rd, 2018 and December 19th, 2021 (last day included in the data collection process). Even though the channel existed before the outbreak of the COVID-19 pandemic,

an exploration of message frequencies clearly reveals that the channel gained relevant momentum at the beginning of the pandemic in Germany in spring 2020, as can be seen in figure 4.1.

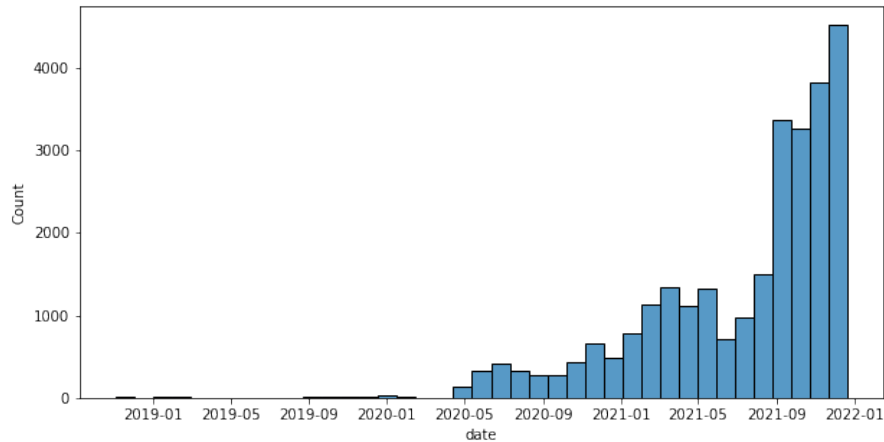


Figure 4.1: Message frequencies

4.1.3 Text cleaning

The initial data collection yielded 31.430 rows of data. After the removal of text duplicates and rows not containing any text, 27.627 rows remained for further processing.

@user_mentions or #hashtags were deliberately included, and the special characters and words of these tokens were not split up into separate tokens. In-word hyphens (such as in Demo-Termine) were also not split up from their surrounding words. For this, the tokenizer of the German medium-size Spacy model `de\_core\_news\_md` was applied with a customized token match pattern as shown in the following code snippet:

```
import re
import spacy
from spacy.tokenizer import _get_regex_pattern
```

```

model="de_core_news_md"
nlp = spacy.load(model)

# code to keep special tokens as a sequence, taken from:
# https://stackoverflow.com/questions/43388476/how-could-spacy-tokenize-hashtag-as-a-whole
# get default pattern for tokens that don't get split
re_token_match = _get_regex_pattern(nlp.Defaults.token_match)
# add patterns (here: #hashtags, in-word hyphens, @mentions)
re_token_match = f"({re_token_match})|#\w+|\w+-\w+|@\w+"
# overwrite token_match function of the tokenizer
nlp.tokenizer.token_match = re.compile(re_token_match).match

```

Preventing the special characters in hashtags and user mentions from being split up was intended to serve for creating a realistic social media corpus and allow to include these special characters into the labels assigned during the annotation process. However, evaluation of training results indicate that it might indeed make sense to split up these special characters and simply treat them as separate tokens which are not treated as part of the tagged named entities.

The overall text cleaning process was restricted to an absolute minimum in order to create a sample data which is as realistic as possible with regard to the task of recognizing named entities in social media text. This basically included the removal of newlines and splitting up of compound hashtags into separate hashtags (so that e.g. #food#delicious would be split up into two different tokens #food and #delicious). The following code demonstrates the respective steps, including the application of the customized tokenizer.

```

import re
import spacy
from spacy.tokenizer import _get_regex_pattern

```

```
def preprocess_text(s, nlp):
    # remove newlines
    s = ' '.join(s.split())
    # split up compound hashtags, e.g. #food#delicious
    s = re.sub(r'##', r' #', s)
    doc = nlp(s)
    # tokenize keeping all tokens, no removal of emojis
    words = [token.text for token in doc]
    # join tokens back together
    result = " ".join(words)
    #replace duplicate whitespaces
    result = re.sub(r'  ', ' ', result)
    return result
```

When applying the `spacy` tokenizer, not only alphabetic tokens but all tokens were obtained. This was due to the fact that tokens representing dates and times usually consist of numbers and punctuation. Thus, in order to include dates and times in the sample data, all tokens were retained. URLs and emojis were also included in the final data, and the capitalization of tokens was retained as well.

The above code parts were then applied to all texts in the dataset, yielding a new column including all the cleaned text values:

```
from datetime import datetime

ts_before = datetime.now()
df['cleaned_text'] = df['text'].apply(lambda x:
                                     preprocess_text(x, nlp) if len
                                     (x) > 0 else None)

ts_after = datetime.now()
print(f"Took {ts_after - ts_before}")
print(len(df), "rows remaining.")
```

Output:


```
Took 0:03:43.779192  
27627 rows remaining.
```

After this step, rows not containing values for cleaned text were removed from the dataset, leaving 22.803 messages for sample selection.

4.1.4 Sample Selection

For sample selection, first the text length for each text was obtained to enable filtering texts based on their length. The second step consisted of detecting the language for each text in the dataset in order to be able to filter out non-German texts. For this, the Python package `langdetect` was applied to the cleaned texts.

After performing language detection on all messages in the channel, results show that German-language content is clearly predominant with $\sim 83\%$ (22.803 messages), followed by English ($\sim 3.8\%$, 3.823 messages), and then French, Italian, and Dutch (1% each).

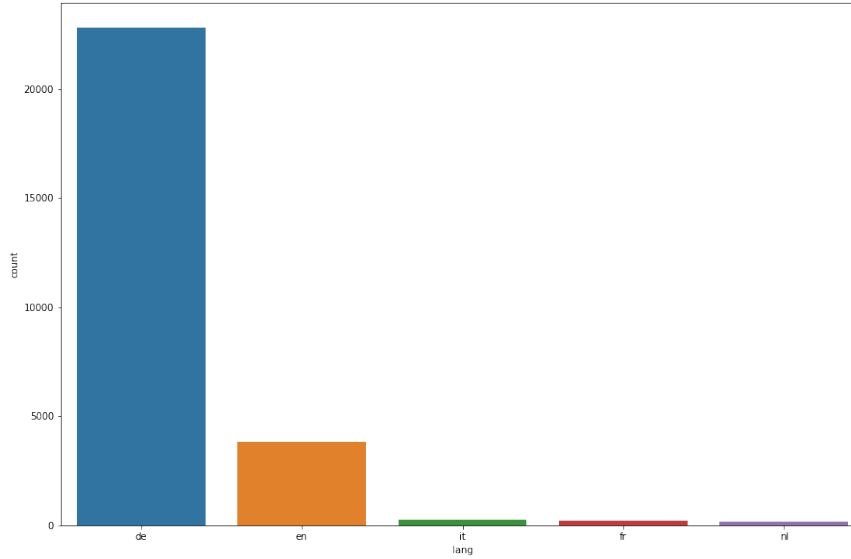


Figure 4.2: Five most frequent languages

It should be noted that the language detection was sometimes flawed due to the fact that the channel contains a number of mixed-language texts resulting in messages being classified as German while actually also consisting of a relevant fraction of e.g. English or other non-German languages. These mixed-language messages were filtered out later during the annotation process described in section However it could be argued that they should be included, since they represent realistic social media data and NER systems should be able to handle such kinds of texts as well.

As a preparatory step for the final sample selection, the dataset was reduced to only German-language texts (22.803 messages remaining). Out of these messages, only those with a minimum length of 100 characters (in the `cleaned_text` column) were selected (14.605 messages remaining).

An exploration of the text length of all German-language messages shows

that the messages have a median text length of 207 characters, and 75% of the messages have a text length of up to 400 characters. The minimum text length is 5 characters, the longest text consists of 4458 characters.

text_length	
count	24001.0
mean	360.0
std	456.0
min	5.0
25%	125.0
50%	207.0
75%	403.0
max	4458.0

Figure 4.3: Descriptive statistiscs for text length of all German-language messages in dataset

Considering these insights, only messages consisting of at least 100 characters were selected (20.105 messages remaining). Out of these, twenty percent of the messages were randomly selected (4021 messages remaining).

Insights from prior research with Telegram data in the context of the project Digitaler Hass revealed that channels often contain a relatively high number of similar messages. To avoid redundancy in the sample, the similarity of the remaining messages was computed based on the **Levenshtein** distance:

```
import Levenshtein as lev
```

```

import pandas as pd
from datetime import datetime
from itertools import combinations

def get_levenshtein_results(df, column='cleaned_text'):
    lev_df = pd.DataFrame()
    new_df = pd.DataFrame(combinations(df[column],2), columns
                            =["text_1", "text_2"])
    new_df["lev_score"] = new_df.apply(lambda x: lev.ratio(x[
        0],x[1]), axis=1)
    lev_df = pd.concat([lev_df, new_df], axis=0).reset_index(
        drop=True)
    return lev_df

def get_message_ids(df, column='message_id'):
    lev_df_ids = pd.DataFrame()
    new_df_ids = pd.DataFrame(combinations(df[column],2),
                              columns=["message_id_1", "
                              message_id_2"])
    lev_df_ids = pd.concat([lev_df_ids, new_df_ids], axis=0).
        reset_index(drop=True)
    ts_after = datetime.now()
    return lev_df_ids

def combine_levenshtein_results_with_message_ids(lev_df,
        lev_df_ids):
    levs = pd.DataFrame()
    levs = pd.concat([lev_df,lev_df_ids], axis=1)
    ts_after = datetime.now()
    return levs

def remove_similar_messages(lev_ids_df, sample_df, threshold):
    second_ids = lev_ids_df[lev_ids_df['lev_score'] >=

```

```

                                threshold].message_id_2.
                                tolist()
ids_to_drop = list(set(second_ids))
result_df = sample_df[~sample_df.message_id.isin(
                                ids_to_drop)]
return result_df

```

A threshold of 0.8 was defined. If the Levensthein score between for a pair of messages was greater than or equal to this threshold, the second message was removed from the sample.

The remaining sample after this step consisted of 3.095 messages.

In a second iteration, an additional sample of short texts was selected. This was supposed to reflect the fact that social media texts can be of very short length and thus should be included in the training data for NER systems. The sampling process for these short texts was similar to the first sampling iteration, except for the sampling size which was 50% instead of 20% in the first iteration. After random sampling and removing similar messages based on the Levenshtein score, a sample of 540 short messages remained and was added to the overall sample dataset.

4.2 Annotation scheme

Frechheit - WNUT 15:

annotation schemes vary greatly in terms of: granularity, types of entities, detailedness of instructions and definitions

labelstudio

IOB encoding: each word either begins, is inside, or is outside of a named entity

Loosely based on Benikova, Biemann, and Reznicek 2014, LDC 2005, Chinchor 1997, Schön et al. 2018

4.3 Annotated dataset

descriptive statistics max sequence length number of documents number of tokens per entity type

4.4 Baseline

Lit: Ritter et al. /Baldwin?

Create Baseline with XLM Roberta

4.5 Models

Experiments on smartdata, and Telegram separately: - DistilBERT - RoBERTa base - gbert base - dbmdz

selected: gbert and roberta base (best validation loss and F1)

4.6 Building a Pipeline

pipeline groups together three steps: preprocessing, passing the inputs through the model, and postprocessing

Like other neural networks, Transformer models can't process raw text directly, so the first step of our pipeline is to convert the text inputs into numbers that the model can make sense of. To do this we use a tokenizer, which will be responsible for:

- Splitting the input into words, subwords, or symbols (like punctuation) that are called tokens
- Mapping each token to an integer
- Adding additional inputs that may be useful to the model

Table 4.1: Results from fine-tuning results different models on Telegram dataset

Model	Validation Loss	F1 score
dbmdz/bert-base-german-uncased	0.2285	0.7440

4.6.1 Tokenization

All this preprocessing needs to be done in exactly the same way as when the model was pretrained

use Autotokenizer from_pretrained

Once we have the tokenizer, we can directly pass our sentences to it and we ll get back a dictionary that s ready to feed to our model

Subword tokenization, alignment

4.6.2 Vectorization?

convert the list of input IDs to tensors.

Transformer models only accept tensors as input. If this is your first time hearing about tensors, you can think of them as NumPy arrays instead. A NumPy array can be a scalar (0D), a vector (1D), a matrix (2D), or have more dimensions. It's effectively a tensor; other ML frameworks' tensors behave similarly, and are usually as simple to instantiate as NumPy arrays.

To specify the type of tensors we want to get back (PyTorch, TensorFlow, or plain NumPy), we use the `return_tensors` argument:

padding, truncation, `return_tensors` erklären können

Don't worry about padding and truncation just yet; we'll explain those later. The main things to remember here are that you can pass one sentence or a list of sentences, as well as specifying the type of tensors you want to get back (if no type is passed, you will get a list of lists as a result).

The output itself is a dictionary containing two keys, `input_ids` and `attention_mask`. `input_ids` contains two rows of integers (one for each sentence) that are the unique identifiers of the tokens in each sentence. We'll explain what the `attention_mask` is later in this chapter.

outputs

`checkpoint / pretrained` This architecture contains only the base Transformer module: given some inputs, it outputs what we'll call hidden states, also known as features. For each model input, we'll retrieve a high-dimensional vector representing the contextual understanding of that input by the Transformer model. While these hidden states can be useful on their own, they're usually inputs to another part of the model, known as the head.

The vector output by the Transformer module is usually large. It generally has three dimensions:

Batch size: The number of sequences processed at a time (2 in our example). Sequence length: The length of the numerical representation of the sequence (16 in our example). Hidden size: The vector dimension of each model input.

Note that the outputs of Transformers models behave like `namedtuples` or dictionaries. You can access the elements by attributes (like we did) or by key (`outputs["last_hidden_state"]`), or even by index if you know exactly where the thing you are looking for is (`outputs[0]`).

4.6.3 Model heads

The model heads take the high-dimensional vector of hidden states as input and project them onto a different dimension. They are usually composed of one or a few linear layers. The output of the Transformer model is sent directly to the model head to be processed.

In this diagram, the model is represented by its embeddings layer and the subsequent layers. The embeddings layer converts each input ID in the tokenized input into a vector that represents the associated token. The subsequent layers manipulate those vectors using the attention mechanism to produce the final representation of the sentences.

There are many different architectures available in Transformers, with each one designed around tackling a specific task.

here: for token classification (NER)

4.6.4 Post-processing

The values we get as output from our model don't necessarily make sense by themselves

Those are not probabilities but logits, the raw, unnormalized scores outputted by the last layer of the model. To be converted to probabilities, they need to go through a SoftMax layer (all Transformers models output the logits, as the loss function for training will generally fuse the last activation function, such as SoftMax, with the actual loss function, such as cross entropy)

To get the labels corresponding to each position, we can inspect the `id2label` attribute of the model config

```
model.config.id2label
```

4.7 Evaluation

- training loss - validation loss - early stopping - F1 - precision, recall, accuracy erklären

Overfitting

Error Analysis

Confusion matrix

4.8 Hyperparameter tuning

4.9 Processing of results

4.10 Summarization

Chapter 5

Results

Chapter 6

Conclusion

6.1 Summarization

6.2 Discussion

more data

improved annotation scheme (zb () / nicht taggen, # splitten), wie umgehen mit hashtags für Demos zB #b2908

entity disambiguation

relationship tagging

future work -i main adaptation on large telegram corpus

Bibliography

- Aggarwal, Charu C. (2018). *Machine Learning for Text*. en. Cham: Springer International Publishing. ISBN: 978-3-319-73530-6 978-3-319-73531-3. DOI: 10.1007/978-3-319-73531-3. URL: <http://link.springer.com/10.1007/978-3-319-73531-3> (visited on 05/14/2022).
- Akbik, Alan, Blythe, Duncan, and Vollgraf, Roland (2018). “Contextual String Embeddings for Sequence Labeling”. In: *COLING 2018, 27th International Conference on Computational Linguistics*, pp. 1638–1649.
- (2022a). *flair/ner-english* · Hugging Face. (Accessed on 07/23/2022). URL: <https://huggingface.co/flair/ner-english>.
- (2022b). *flair/ner-german* · Hugging Face. (Accessed on 07/23/2022). URL: <https://huggingface.co/flair/ner-german>.
- Akbik, Alan et al. (2019). “FLAIR: An easy-to-use framework for state-of-the-art NLP”. In: *NAACL 2019, 2019 Annual Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pp. 54–59.
- Baldwin, Timothy et al. (July 2015). “Shared Tasks of the 2015 Workshop on Noisy User-generated Text: Twitter Lexical Normalization and Named Entity Recognition”. In: *Proceedings of the Workshop on Noisy User-generated Text*. Beijing, China: Association for Computational Linguistics, pp. 126–135. DOI: 10.18653/v1/W15-4319. URL: <https://aclanthology.org/W15-4319> (visited on 05/16/2022).

- Benikova, Darina, Biemann, Chris, and Reznicek, Marc (May 2014). “NoStandard Named Entity Annotation for German: Guidelines and Dataset”. In: *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC’14)*. Reykjavik, Iceland: European Language Resources Association (ELRA), pp. 2524–2531. URL: http://www.lrec-conf.org/proceedings/lrec2014/pdf/276_Paper.pdf.
- Benikova, Darina et al. (2014). “Germeval 2014 named entity recognition: Companion paper”. In: *Proceedings of the KONVENS GermEval Shared Task on Named Entity Recognition, Hildesheim, Germany*, pp. 104–112.
- Chinchor, Nancy (Sept. 1997). *MUC-7 Named Entity Task Definition*. (Accessed on 07/23/2022). URL: https://www-nlpir.nist.gov/related_projects/muc/proceedings/ne_task.html.
- Chinchor, Nancy and Sundheim, Beth (Aug. 2003). *Message Understanding Conference (MUC) 6*. Artwork Size: 10240 KB Pages: 10240 KB Type: dataset. DOI: 10.35111/WBCC-Y063. URL: <https://catalog.ldc.upenn.edu/LDC2003T13> (visited on 06/27/2022).
- Collobert, Ronan et al. (2011). “Natural Language Processing (Almost) from Scratch”. en. In: *NATURAL LANGUAGE PROCESSING*, p. 45.
- Derczynski, Leon et al. (Mar. 2015). “Analysis of named entity recognition and linking for tweets”. en. In: *Information Processing & Management* 51.2, pp. 32–49. ISSN: 03064573. DOI: 10.1016/j.ipm.2014.10.006. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0306457314001034> (visited on 06/27/2022).
- Derczynski, Leon et al. (Sept. 2017). “Results of the WNUT2017 Shared Task on Novel and Emerging Entity Recognition”. In: *Proceedings of the 3rd Workshop on Noisy User-generated Text*. Copenhagen, Denmark: Association for Computational Linguistics, pp. 140–147. DOI: 10.18653/v1/W17-4418. URL: <https://aclanthology.org/W17-4418> (visited on 07/21/2022).

- Ethnologue (May 2022). *How many languages are there in the world?* en. URL: <https://www.ethnologue.com/guides/how-many-languages> (visited on 06/19/2022).
- ExplosionAI (2022a). *English · spaCy Models Documentation*. (Accessed on 07/23/2022). URL: https://spacy.io/models/en#en_core_web_md.
- (2022b). *EntityRecognizer · spaCy API Documentation*. (Accessed on 07/23/2022). URL: <https://spacy.io/api/entityrecognizer>.
- (2022c). *Facts & Figures · spaCy Usage Documentation*. en. URL: <https://spacy.io/usage/facts-figures> (visited on 07/23/2022).
- (2022d). *German · spaCy Models Documentation*. (Accessed on 07/23/2022). URL: https://spacy.io/models/de#de_core_news_md.
- (2022e). *TransitionBasedParser · spaCy API Documentation*. (Accessed on 07/23/2022). URL: <https://spacy.io/api/architectures#TransitionBasedParser>.
- Grishman, Ralph and Sundheim, Beth (1996). “Message Understanding Conference-6: A Brief History”. In: *COLING 1996 Volume 1: The 16th International Conference on Computational Linguistics*. URL: <https://aclanthology.org/C96-1079> (visited on 06/27/2022).
- Jurafsky, Daniel and Martin, James H. (2021). *Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.
- LDC (2005). *ACE (Automatic Content Extraction) English Annotation Guidelines for Entities version 6.6*. (Accessed on 07/23/2022). URL: <https://www ldc.upenn.edu/sites/www ldc.upenn.edu/files/english-entities-guidelines-v6.6.pdf>.
- Magueresse, Alexandre, Carles, Vincent, and Heetderks, Evan (June 2020). *Low-resource Languages: A Review of Past Work and Future Challenges*. Number: arXiv:2006.07264 arXiv:2006.07264 [cs]. URL: <http://arxiv.org/abs/2006.07264> (visited on 06/19/2022).

- Mikolov, Tomas et al. (2013). *Efficient Estimation of Word Representations in Vector Space*. DOI: 10.48550/ARXIV.1301.3781. URL: <https://arxiv.org/abs/1301.3781>.
- Nadeau, David and Sekine, Satoshi (Aug. 2007). “A survey of named entity recognition and classification”. en. In: *Linguisticae Investigationes* 30.1, pp. 3–26. ISSN: 0378-4169, 1569-9927. DOI: 10.1075/li.30.1.03nad. URL: <http://www.jbe-platform.com/content/journals/10.1075/li.30.1.03nad> (visited on 07/04/2022).
- Nothman, Joel et al. (2013). “Learning multilingual named entity recognition from Wikipedia”. In: *Artificial Intelligence* 194, pp. 151–175. ISSN: 0004-3702. DOI: <https://doi.org/10.1016/j.artint.2012.03.006>. URL: <https://www.sciencedirect.com/science/article/pii/S0004370212000276>.
- Pennington, Jeffrey, Socher, Richard, and Manning, Christopher D. (2014). “GloVe: Global Vectors for Word Representation”. In: *Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543. URL: <http://www.aclweb.org/anthology/D14-1162>.
- Ralph Weischedel Martha Palmer, Mitchell Marcus Eduard Hovy Sameer Pradhan Lance Ramshaw Nianwen Xue Ann Taylor Jeff Kaufman Michelle Franchini Mohammed El-Bachouti Robert Belvin Ann Houston (Oct. 2013). *OntoNotes Release 5.0 - Linguistic Data Consortium*. (Accessed on 07/23/2022). URL: <https://catalog.ldc.upenn.edu/LDC2013T19>.
- Ritter, Alan et al. (2011). “Named Entity Recognition in Tweets: An Experimental Study”. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. EMNLP ’11. Edinburgh, United Kingdom: Association for Computational Linguistics, 1524–1534. ISBN: 9781937284114.
- Schiersch, Martin et al. (2018). “A German Corpus for Fine-Grained Named Entity Recognition and Relation Extraction of Traffic and Industry Events”. In: *Proceedings of the Eleventh International Conference on Language Re-*

- sources and Evaluation (LREC 2018)*. Ed. by Nicoletta Calzolari (Conference chair) et al. Miyazaki, Japan: European Language Resources Association (ELRA). ISBN: 979-10-95546-00-9.
- Schweter, Stefan and Akbik, Alan (2020). *FLERT: Document-Level Features for Named Entity Recognition*. arXiv: 2011.06993 [cs.CL].
- Schön, Saskia et al. (2018). *DFKI Annotation Guidelines, Version 1.0*. (Accessed on 07/23/2022). URL: https://github.com/DFKI-NLP/smartdata-corpus/blob/master/SmartData_Corpus_Annotation_Guidelines_Feb_2018_v1.0.pdf.
- Singh, Anil Kumar (2008). “Natural Language Processing for Less Privileged Languages: Where do we come from? Where are we going?” In: *Proceedings of the IJCNLP-08 Workshop on NLP for Less Privileged Languages*. URL: <https://aclanthology.org/I08-3004> (visited on 06/19/2022).
- Strauss, Benjamin et al. (Dec. 2016). “Results of the WNUT16 Named Entity Recognition Shared Task”. In: *Proceedings of the 2nd Workshop on Noisy User-generated Text (WNUT)*. Osaka, Japan: The COLING 2016 Organizing Committee, pp. 138–144. URL: <https://aclanthology.org/W16-3919> (visited on 07/21/2022).
- Tjong Kim Sang, Erik F. (2002). “Introduction to the CoNLL-2002 Shared Task: Language-Independent Named Entity Recognition”. In: *COLING-02: The 6th Conference on Natural Language Learning 2002 (CoNLL-2002)*. URL: <https://aclanthology.org/W02-2024>.
- Tjong Kim Sang, Erik F. and De Meulder, Fien (2003). “Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition”. In: *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pp. 142–147. URL: <https://aclanthology.org/W03-0419> (visited on 06/19/2022).
- Tunstall, Lewis et al. (2022). *Natural language processing with Transformers: building language applications with Hugging Face*. eng. First edition. Bei-

- jing Boston Farnham Sebastopol Tokyo: O'Reilly. ISBN: 978-1-09-810324-8.
- Vajjala, Sowmya et al. (2020). *Practical natural language processing: a comprehensive guide to building real-world NLP systems*. First edition. OCLC: on1125266646. Sebastopol, CA: O'Reilly Media. ISBN: 978-1-4920-5405-4.
- Weichbrodt, Gregor and Stanjek, Grischa (2022). *Teledash – analysis and research software for Telegram*. URL: <https://github.com/democ-de/teledash>.
- Yadav, Vikas and Bethard, Steven (Oct. 2019). *A Survey on Recent Advances in Named Entity Recognition from Deep Learning models*. Tech. rep. arXiv:1910.11470. arXiv:1910.11470 [cs] type: article. arXiv. URL: <http://arxiv.org/abs/1910.11470> (visited on 05/14/2022).
- Zong, Chengqing, Xia, Rui, and Zhang, Jiajun (2021). “Information Extraction”. en. In: *Text Data Mining*. Singapore: Springer Singapore, pp. 227–283. ISBN: 9789811600999 9789811601002. DOI: 10.1007/978-981-16-0100-2_10. URL: https://link.springer.com/10.1007/978-981-16-0100-2_10 (visited on 05/14/2022).

Appendix

List of Figures

2.1	Examples of noisy text in tweets, taken from Ritter et al. 2011	15
2.2	Lexical variations of the word ‘tomorrow’ in tweets, taken from Ritter et al. 2011	17
4.1	Message frequencies	41
4.2	Five most frequent languages	45
4.3	Descriptive statistiscs for text length of all German-language messages in dataset	46

List of Tables

3.1	Detected entities for German-language example messages . . .	33
3.2	Overview of spaCy’s entity types	35
3.3	Detected entities for English-language example messages . . .	38
4.1	Results from fine-tuning results different models on Telegram dataset	50
1	Definition of Entity types	68

List of Acronyms

Annotation guidelines

The following annotation guidelines are a combined adaptation of Benikova, Biemann, and Reznicek 2014, LDC 2005, Chinchor 1997, and Schön et al. 2018.

.1 Named Entity - Basic Definition

Names are denominators for a single unique object in the real world. Only full noun phrases are considered as named entities. Pronouns and all other phrases can be ignored. Phrases only partly containing names (e.g. ‘deutschlandweit’), or adjectives referring to entities (‘euklidisch’) can be ignored.

Only one entity type per token should be annotated. In case of multiple options (e.g. ‘Charité’ could be referred to as organization or as location, same for ‘Landtag’ or ‘Bundestag’), the context of usage should be considered for the final classification.

Named entities can consist of more than one token. The initial token of an entity should be tagged with the B-label of the corresponding entity type. The following tokens belonging to the same entity should be labelled with the I-label of the same type. For

Combined named entities including different entity types (e.g. ‘Wien-Demo’) should be tagged separately if possible (Wien[LOC]-Demo[ACTION]), or else tagged as the dominant entity type (here: [ACTION]) to avoid nested entities.

Whitespace should not be labelled as part of an entity. Token separators such as /, - or . should be included if they are inside an entity sequence, and excluded if they separate two different entities, or an entity and a non-entity token. Hashtags should be included in the labeled span.

Dates and times can be represented by numbers (31.10.2019, 16:00), and strings ('Morgen', 'Uhr').

Emoticons, user mentions and (parts of) urls should not be tagged as named entities.

.2 Definition of Different Entity Types

For annotation, the the 6 entity types PERSON, LOCATION, ORGANIZATION, DATE, TIME, and ACTION are used. The label set consists of B-tag and I-tag for each entity type, resulting in a total of 12 labels.

reftab:entity_{type}sshowsthecoredefinitionsandexamplesfor

.3 Specifications

.3.1 Mixed-language messages

Skip mixed-language in which the actual message is non-German language.

Examples:

- *We are angry and pissed off - Raus auf die Straße Demotermine!*
- *Brilliant numbers in Carlisle ! Raus auf die Straßen @Demotermine*
- *MASSIMA DIFFUSIONE <http://t.me/stopdittatura/1930> Raus auf die Straßen @Demotermine*

Table 1: Definition of Entity types

Label	Description	Examples
PER	Named, single individual, limited to humans. Fictional persons can be included, as long as they're referred to by name.	Karl Lauterbach, Merkel, Harry Potter, Kretschmer, Karl Hilz, Markus
LOC	Names of politically or geographically defined locations, including countries, provinces, regions, cities, districts, addresses including streets, squares, and postal codes, (named) buildings or other man-made structures, shopping centers, parking lots, or restaurants, cardinal points. No metaphorical locations.	Deutschland, Frankreich, #BW, Berlin, Bundestag, Restaurant Walliserkanne, Straße des 17. Juni, Neptunbrunnen, Alex, Nollendorfsplatz, Norden, Süden, Hellersdorf
ORG	A named corporate, governmental, or other organizational entity: Organization entities can be corporations, agencies, institutions, newspapers, tv/radio channels, other media platforms, political parties, non-governmental organizations, and political groups with a formal or informal organizational structure.	ARD, Polizei Berlin, Youtube, Telegram, Amadeu Antonio Stiftung, Querdenken711, Antifa, Bundesregierung, Gesundheitsamt, Französische Polizei, Facebook, Google, Bill Gates Foundation, KSV LiLi, FC St. Pauli, SGD-Ultras, Bündnis21 Hessen, Polizei, Polizeipräsidium Pforzheim
DATE	Absolute or relative date expressions	23.09.2022, 20210930, 3108, Morgen, Heute, 2019, #Sa2011
TIME	Specific, absolute time expressions referring to times of day, and time codes.	13:10, 13:12 Uhr, 16 Uhr, 6:49
ACTION	A specific instance of an action or event undertaken to express political beliefs, e.g. demonstrations, vigils, motorcades, walks in public.	Kundgebung, Demo, Demonstration, Spaziergang, Aufzug, Aufzug Ruffer Trommeln

.3.2 Incomprehensible messages

Skip incomprehensible messages.

Example: *“Sterne-Restaurant Walliserkanne . Naudreck . Bersets faule Tricks . Uniarzt panikfrei . Herzmuskel . Neuer Beitrag #Walliserkanne Raus auf die Straßen Demotermine! Übersicht / Overview”*

.3.3 Short messages

Skip short messages which do not include at least two named entities.

Examples:

- *“AnwaelteFuerAufklaerung Demotermine”*
- *“Egal wo ihr diesen , oder die kommenden Samstage seid ! RheinCandleLight Demotermine”*

.3.4 Punctuation

Punctutaion should only be included in the tag if they appear in the middle of sequence representing an entity.

Example: *Donnerstag[B-DATE] ,[I-DATE] 17.04.2020[I-DATE]*

Hashtags

If a hashtag is part of a named entity, it should be included in the tag.

Examples:

- *#Berlin[B-ORG]*
- *#BW[B-ORG]*

- #CH[B-ORG]
- #Hardy[B-PER] Groeneveld[I-PER]
- #Mutigmacher[B-ORG]

If a hashtag is part of a sequence combining two named entities, it should be included in the tag for the first entity.

Example: #be[B-LOC]2908[B-DATE]

.4 Detailed Annotation Instructions for Entity Types

.4.1 PERSON

Person names can consist of given name and last name, or only first / only last name. Titles (e.g. Dr.) and forms of address (Herr, Frau) should not be annotated.

Examples:

- *Scholz*[B-PER]
- *Carolyn*[B-PER] *Matthie*[I-PER]
- *Merkel*[B-PER]
- *Anselm*[B-PER] *Lenz*[I-PER]
- *Karl*[B-PER]

.4.2 LOCATION

Do not label derived location words (e.g. *Kreuzberger Nächte* as location).

If a sequence contains a number of tokens representing a location entity (e.g. an address), use the B-tag for the initial token and the I-tag for all succeeding tokens belonging to the same entity, including special characters such as , - . or /. The sequence should be long enough to create a representation of an entity which can be understood without the succeeding tokens.

Examples:

- *Straße[B-LOC] des[I-LOC] 17.[I-LOC] Juni[I-LOC]*
- *Bad[B-LOC] Oldesloe[I-LOC]*
- *Elbe[B-LOC] Elster[I-LOC]*
- *Bremen[B-LOC]-Vegesack[I-LOC]*
- *Lutherstadt[B-LOC] Wittenberg[I-LOC]*
- *Landeplatz[B-LOC] Grißkirchen[I-LOC]*
- *Google[B-LOC] center[I-LOC]*
- *Stuttgarter[B-LOC] Landtag[I-LOC]*
- NOTE: *Berlin[B-LOC] Neptunbrunnen[B-LOC]*

If the sequence contains a specification of the location, include the specification using the I-tags.

Examples:

- *Königstraße[B-LOC] Richtung[I-LOC] HBF[I-LOC]*
- *Bernauer[B-LOC] Str[I-LOC] Richtung[I-LOC] Süden[I-LOC]*

.4. DETAILED ANNOTATION INSTRUCTIONS FOR ENTITY TYPES⁷²

- *Marktoberdorf[B-LOC]-Kaufbeuren[I-LOC]-Neugablonz[I-LOC]*
- *Bremen[B-LOC]-Vegesack[I-LOC]*
- *D-67454[B-LOC] Haßloch[I-LOC]*

.4.3 ORGANIZATION

If a sequence contains a number of tokens representing an organization entity, use the B-tag for the initial token and the I-tag for all succeeding tokens belonging to the same entity, including special characters such as , - . or /. The sequence should be long enough to create a representation of an entity which can be understood without the succeeding tokens.

Examples:

- *Bundesregierung[B-ORG]*
- *Gesundheitsamt[B-ORG]*
- *Französische[B-ORG] Polizei[I-ORG]*
- *Google[B-ORG]*
- *Bill[B-ORG] Gates[I-ORG] Foundation[I-ORG]*
- *KSV[B-ORG] LiLi[I-ORG]*
- *FC[B-ORG] St.[I-ORG] Pauli[I-ORG]*
- *Bündnis21[B-ORG] Hessen[I-ORG]*
- *Polizeipräsidium[B-ORG] Pforzheim[I-ORG]*

.4.4 TIME

Time expressions can include specific times of day and time codes (usually appearing after URLs. Sequences denoting a time span / duration should be annotated with the B-tag for the initial token and I-tags for the succeeding tokens.

Examples.

- *15:00[B-TIME]*
- *17:00[B-TIME] Uhr[I-TIME]*
- *www.example.com[O] 00:30[B-TIME]*
- *Mittag[B-DATE]*
- *15:30[B-TIME] -[I-TIME] 19:30[I-TIME]*

.4.5 DATE

Date expressions can be numeric values or strings. Sequences denoting a date spanshould be annotated with the B-tag for the initial token and I-tags for the succeeding tokens.

Examples:

- *2019[B-DATE]*
- *14.12.21[B-DATE]*
- *2.7.21[B-DATE]*
- *31. [B-DATE] JULI[I-DATE]*
- *01122020[B-DATE]*

.4. DETAILED ANNOTATION INSTRUCTIONS FOR ENTITY TYPES⁷⁴

- *Pfingstmontag*[B-DATE]
- *HEUTE*[B-DATE]
- *3108*[B-DATE]
- *201905*[B-DATE]
- *2019/03/05*[B-DATE]
- *28.* [B-DATE]+ [I-DATE] *29.8.* [I-DATE]
- *17042021*[B-DATE] –[I-DATE] *21042021*[I-DATE]

.4.6 ACTION

Actions represent unique real-world events or actions. They must not necessarily be named, but be identifiable as unique object in the context in which they are mentioned. Also, the type of action should be expressed.

Examples:

- *Angriff*[B-ACTION]
- *Demo-B*
- *Autokorso*[B-ACTION]
- *Offenes*[B-ACTION] *Treffen*[I-ACTION]
- *Aufzug*[B-ACTION] *RufderTrommeln*[I-ACTION]
- *Zug*[B-ACTION]
- *Plakate*[B-ACTION]
- *Infostände*[B-ACTION]

.4. DETAILED ANNOTATION INSTRUCTIONS FOR ENTITY TYPES⁷⁵

- *Informationstouren*[B-ACTION]
- *Anti*[B-ACTION]-*Corona*[I-ACTION]-*Proteste*[I-ACTION]
- *Protestzug*[B-ACTION]
- *Spaziergänge*[B-ACTION]
- *Querdenker*[B-ORG]-*Demo*[B-ACTION]
- *Korso*[B-ACTION] *Nord*[I-ACTION]
- *Gedenken*[B-ACTION]
- *Aktionswoche*[B-ACTION]
- *Wahlkampf*[B-ACTION]
- *Live-Stream*[B-ACTION]

Statutory Declaration

I herewith formally declare that I have written the submitted thesis independently. I did not use any outside support except for the quoted literature and other sources mentioned in the paper.

I clearly marked and separately listed all of the literature and all of the other sources which I employed when producing this academic work, either literally or in content.

I am aware that the violation of this regulation will lead to the failure of the thesis.

.....
Name Signature

.....
Matriculation Number Place and Date